

Les 23 Design Patterns GoF

Explications progressives + 23 démonstrations Java autonomes et compilables

Chaque bloc peut être copié tel quel dans le fichier indiqué, puis compilé et exécuté.

Correction importante concernant Prototype : dans cette version, chaque méthode utilisée est réellement présente dans le code. Par exemple, setTitle() est déclarée dans Document. Les 23 démonstrations ont été compilées et exécutées avec OpenJDK 21.

Comment utiliser les exemples

- Copiez le bloc complet du pattern dans un fichier portant exactement le nom indiqué, par exemple StrategyDemo.java.
- Compilez avec : javac StrategyDemo.java
- Exécutez avec : java StrategyDemo
- Chaque fichier contient une seule classe public ; les interfaces et classes auxiliaires sont imbriquées dans cette classe afin que rien d'autre ne soit nécessaire.
- Aucun framework, aucune bibliothèque externe et aucun package ne sont requis.

Les 23 patterns par famille

Création	Structure	Comportement
Factory Method	Adapter	Chain of Responsibility
Abstract Factory	Bridge	Command
Builder	Composite	Interpreter
Prototype	Decorator	Iterator
Singleton	Facade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Template Method
		Visitor

Patterns de création

1. Factory Method - FactoryMethodDemo.java

Idée. Déléguer la création d'un objet concret à une sous-classe, tout en faisant travailler le reste du code avec une abstraction.

Le problème. Vous devez envoyer des notifications, mais le type concret peut varier : e-mail ou SMS. Si le code métier fait lui-même tous les new, il devient directement couplé aux classes concrètes.

Les rôles. Notification = produit abstrait ; EmailNotification/SmsNotification = produits concrets ; NotificationCreator = créateur abstrait ; EmailCreator/SmsCreator = créateurs concrets.

Ce qui se passe. Le client choisit un créateur. La méthode notifyUser() demande au créateur de fabriquer une Notification puis utilise uniquement l'interface Notification.

Pourquoi c'est utile. Le code qui utilise la notification n'a pas besoin de connaître la classe concrète créée. Vous pouvez ajouter un PushCreator sans réécrire notifyUser().

Attention. Factory Method ne signifie pas simplement « mettre new dans une méthode ». Le point important est que la décision de création est déléguée à une implémentation spécialisée.

À retenir : Factory Method = « quel produit concret dois-je créer ? ».

Code complet à copier

```
public class FactoryMethodDemo {

    interface Notification {
        void send(String message);
    }

    static class EmailNotification implements Notification {
        public void send(String message) {
            System.out.println("EMAIL : " + message);
        }
    }

    static class SmsNotification implements Notification {
        public void send(String message) {
            System.out.println("SMS : " + message);
        }
    }

    static abstract class NotificationCreator {
        public abstract Notification createNotification();

        public void notifyUser(String message) {
            Notification notification = createNotification();
            notification.send(message);
        }
    }

    static class EmailCreator extends NotificationCreator {
        public Notification createNotification() {
            return new EmailNotification();
        }
    }

    static class SmsCreator extends NotificationCreator {
        public Notification createNotification() {
            return new SmsNotification();
        }
    }

    public static void main(String[] args) {
        NotificationCreator creator = new EmailCreator();
        creator.notifyUser("Votre commande est validee.");

        creator = new SmsCreator();
```

```
        creator.notifyUser("Votre colis est parti.");  
    }  
}
```

Résultat obtenu à l'exécution

EMAIL : Votre commande est validee.
SMS : Votre colis est parti.

2. Abstract Factory - AbstractFactoryDemo.java

Idée. Créer une famille complète d'objets compatibles sans exposer leurs classes concrètes.

Le problème. Votre application doit fonctionner soit avec AWS, soit avec Azure. Dans chaque cas, stockage et base de données doivent appartenir à la même famille technologique.

Les rôles. CloudFactory = fabrique abstraite ; AwsFactory/AzureFactory = fabriques concrètes ; Storage/Database = produits abstraits ; AwsStorage, AwsDatabase, AzureStorage, AzureDatabase = produits concrets.

Ce qui se passe. Application reçoit une CloudFactory. Elle demande createStorage() et createDatabase(), puis travaille seulement avec Storage et Database.

Pourquoi c'est utile. Vous changez tout un environnement cohérent en remplaçant une seule fabrique. Application reste inchangée.

Attention. La différence avec Factory Method tient surtout à la notion de famille : Abstract Factory fournit plusieurs types de produits qui vont ensemble.

À retenir : Abstract Factory = « quelle famille d'objets dois-je utiliser ? ».

Code complet à copier

```
public class AbstractFactoryDemo {

    interface Storage {
        void save(String fileName);
    }

    interface Database {
        void connect();
    }

    static class AwsStorage implements Storage {
        public void save(String fileName) {
            System.out.println("AWS S3 stocke : " + fileName);
        }
    }

    static class AwsDatabase implements Database {
        public void connect() {
            System.out.println("Connexion a AWS DynamoDB");
        }
    }

    static class AzureStorage implements Storage {
        public void save(String fileName) {
            System.out.println("Azure Blob stocke : " + fileName);
        }
    }

    static class AzureDatabase implements Database {
        public void connect() {
            System.out.println("Connexion a Azure Cosmos DB");
        }
    }

    interface CloudFactory {
        Storage createStorage();
        Database createDatabase();
    }
```

```

static class AwsFactory implements CloudFactory {
    public Storage createStorage() {
        return new AwsStorage();
    }

    public Database createDatabase() {
        return new AwsDatabase();
    }
}

static class AzureFactory implements CloudFactory {
    public Storage createStorage() {
        return new AzureStorage();
    }

    public Database createDatabase() {
        return new AzureDatabase();
    }
}

static class Application {
    private final Storage storage;
    private final Database database;

    Application(CloudFactory factory) {
        storage = factory.createStorage();
        database = factory.createDatabase();
    }

    void run() {
        database.connect();
        storage.save("facture.pdf");
    }
}

public static void main(String[] args) {
    CloudFactory factory = new AwsFactory();
    new Application(factory).run();

    System.out.println("--- Changement de famille ---");

    factory = new AzureFactory();
    new Application(factory).run();
}
}

```

Résultat obtenu à l'exécution

```

Connexion a AWS DynamoDB
AWS S3 stocke : facture.pdf
--- Changement de famille ---
Connexion a Azure Cosmos DB
Azure Blob stocke : facture.pdf

```

3. Builder - BuilderDemo.java

Idée. Construire progressivement un objet complexe, avec une API lisible, des valeurs par défaut et une validation finale.

Le problème. Un constructeur avec de nombreux paramètres devient difficile à lire : `new Computer("i7", 32, 1000, true, "RTX...")`. On ne sait plus immédiatement à quoi correspond chaque valeur.

Les rôles. `Computer` = produit final ; `Computer.Builder` = constructeur progressif ; `build()` = étape qui valide puis crée l'objet final.

Ce qui se passe. Vous créez un Builder, renseignez seulement les options utiles, puis appelez `build()`. Le constructeur de `Computer` est privé pour imposer ce chemin de construction.

Pourquoi c'est utile. Le code appelant devient auto-documenté, les paramètres optionnels sont naturels et la validation est centralisée.

Attention. Une fluent API n'est pas automatiquement le pattern Builder, mais c'est une forme très courante de son implémentation en Java.

À retenir : Builder = « comment construire proprement cet objet complexe ? ».

Code complet à copier

```
public class BuilderDemo {

    static class Computer {
        private final String cpu;
        private final int ram;
        private final int storage;
        private final boolean wifi;
        private final String gpu;

        private Computer(Builder builder) {
            this.cpu = builder.cpu;
            this.ram = builder.ram;
            this.storage = builder.storage;
            this.wifi = builder.wifi;
            this.gpu = builder.gpu;
        }

        public String toString() {
            return "Computer{cpu='" + cpu + "', ram=" + ram
                + " Go, storage=" + storage + " Go, wifi=" + wifi
                + ", gpu='" + gpu + "'}";
        }
    }

    static class Builder {
        private String cpu;
        private int ram = 8;
        private int storage = 256;
        private boolean wifi = true;
        private String gpu = "Integre";

        Builder cpu(String cpu) {
            this.cpu = cpu;
            return this;
        }

        Builder ram(int ram) {
            this.ram = ram;
            return this;
        }
    }
}
```

```

    Builder storage(int storage) {
        this.storage = storage;
        return this;
    }

    Builder wifi(boolean wifi) {
        this.wifi = wifi;
        return this;
    }

    Builder gpu(String gpu) {
        this.gpu = gpu;
        return this;
    }

    Computer build() {
        if (cpu == null || cpu.trim().isEmpty()) {
            throw new IllegalStateException("Le CPU est obligatoire");
        }
        if (ram <= 0) {
            throw new IllegalStateException("La RAM doit etre positive");
        }
        return new Computer(this);
    }
}

public static void main(String[] args) {
    Computer pc = new Computer.Builder()
        .cpu("Intel i7")
        .ram(32)
        .storage(1000)
        .gpu("RTX 5070")
        .build();

    System.out.println(pc);
}

```

Résultat obtenu à l'exécution

```
Computer{cpu='Intel i7', ram=32 Go, storage=1000 Go, wifi=true, gpu='RTX 5070'}
```

4. Prototype - PrototypeDemo.java

Idée. Créer un nouvel objet à partir d'un objet déjà configuré, en le copiant.

Le problème. Vous disposez d'un document modèle déjà configuré avec une police et plusieurs sections. Refaire toute sa construction pour chaque client est répétitif.

Les rôles. Document = prototype concret ; copy() = opération de duplication ; le constructeur privé Document(Document source) réalise la copie.

Ce qui se passe. prototype.copy() produit un nouvel objet. Ensuite setTitle() et addSection() modifient uniquement la copie. La liste des sections est elle aussi recopiée avec new ArrayList<>(...), ce qui évite de partager la même liste.

Pourquoi c'est utile. Vous réutilisez une configuration existante et pouvez créer rapidement des variantes indépendantes.

Attention. Dans l'ancien document, setTitle() était utilisée sans être montrée. Ici elle est bien déclarée dans Document. J'utilise volontairement un constructeur de copie plutôt que Cloneable : le pattern Prototype n'impose pas l'API clone() de Java.

À retenir : Prototype = « j'ai déjà un bon modèle : je le copie ».

Code complet à copier

```
import java.util.ArrayList;
import java.util.List;

public class PrototypeDemo {

    static class Document {
        private String title;
        private final String font;
        private final List<String> sections;

        Document(String title, String font, List<String> sections) {
            this.title = title;
            this.font = font;
            this.sections = new ArrayList<String>(sections);
        }

        private Document(Document source) {
            this.title = source.title;
            this.font = source.font;
            this.sections = new ArrayList<String>(source.sections);
        }

        Document copy() {
            return new Document(this);
        }

        void setTitle(String title) {
            this.title = title;
        }

        void addSection(String section) {
            sections.add(section);
        }

        public String toString() {
            return "Document{title='" + title + "', font='" + font
                + "', sections='" + sections + "'}";
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    List<String> baseSections = new ArrayList<String>();
    baseSections.add("Identite du client");
    baseSections.add("Conditions generales");

    Document prototype = new Document(
        "Contrat modele",
        "Arial",
        baseSections
    );

    Document contratDupont = prototype.copy();
    contratDupont.setTitle("Contrat - Client Dupont");
    contratDupont.addSection("Option premium");

    System.out.println("Prototype : " + prototype);
    System.out.println("Copie      : " + contratDupont);
}
}

```

Résultat obtenu à l'exécution

```

Prototype : Document{title='Contrat modele', font='Arial', sections=[Identite du client, Conditions generales]}
Copie      : Document{title='Contrat - Client Dupont', font='Arial', sections=[Identite du client, Conditions generales,
Option premium]}

```

5. Singleton - SingletonDemo.java

Idée. Garantir qu'une classe possède une seule instance accessible de manière contrôlée.

Le problème. Certains services techniques peuvent être conçus comme une ressource unique. L'exemple utilise un Logger afin de montrer le mécanisme sans multiplier les objets.

Les rôles. Logger = singleton ; constructeur privé = interdit new Logger() à l'extérieur ; Holder.INSTANCE = instance unique ; getInstance() = point d'accès.

Ce qui se passe. Le premier appel à getInstance() déclenche le chargement du Holder et donc la création de l'unique Logger. Les appels suivants récupèrent exactement le même objet.

Pourquoi c'est utile. La variante Holder est paresseuse et thread-safe grâce aux garanties d'initialisation des classes Java.

Attention. Singleton peut devenir un état global caché et gêner les tests. Dans les applications modernes, un conteneur de dépendances gère souvent la durée de vie singleton à votre place.

À retenir : Singleton = « une seule instance, partagée ».

Code complet à copier

```
public class SingletonDemo {

    static class Logger {
        private Logger() {
        }

        private static class Holder {
            private static final Logger INSTANCE = new Logger();
        }

        static Logger getInstance() {
            return Holder.INSTANCE;
        }

        void log(String message) {
            System.out.println("[LOG] " + message);
        }
    }

    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();

        logger1.log("Application démarrée");
        logger2.log("Utilisateur connecté");

        System.out.println("Meme instance ? " + (logger1 == logger2));
    }
}
```

Résultat obtenu à l'exécution

```
[LOG] Application démarrée
[LOG] Utilisateur connecté
Meme instance ? true
```

Patterns de structure

6. Adapter - AdapterDemo.java

Idée. Rendre compatible une classe existante avec l'interface attendue par votre application.

Le problème. Votre Checkout attend `PaymentProcessor.pay(double euros)`, mais une ancienne API bancaire expose `makePaymentInCents(int cents)`. Les interfaces ne correspondent pas.

Les rôles. `PaymentProcessor` = cible attendue ; `LegacyBankApi` = service incompatible ; `BankAdapter` = adaptateur ; `Checkout` = client.

Ce qui se passe. Checkout appelle `pay(49.90)`. `BankAdapter` convertit 49,90 euros en 4 990 centimes puis appelle l'ancienne API.

Pourquoi c'est utile. Vous réutilisez un composant existant sans modifier ni le client ni la classe legacy.

Attention. Adapter change la forme de l'interface. Il ne cherche pas principalement à simplifier un sous-système : cela, c'est Facade.

À retenir : Adapter = « ces deux interfaces ne parlent pas la même langue ; je traduis ».

Code complet à copier

```
public class AdapterDemo {

    interface PaymentProcessor {
        void pay(double amountInEuros);
    }

    static class LegacyBankApi {
        void makePaymentInCents(int cents) {
            System.out.println("Ancienne banque : paiement de " + cents + " centimes");
        }
    }

    static class BankAdapter implements PaymentProcessor {
        private final LegacyBankApi legacyBankApi;

        BankAdapter(LegacyBankApi legacyBankApi) {
            this.legacyBankApi = legacyBankApi;
        }

        public void pay(double amountInEuros) {
            int cents = (int) Math.round(amountInEuros * 100);
            legacyBankApi.makePaymentInCents(cents);
        }
    }

    static class Checkout {
        private final PaymentProcessor paymentProcessor;

        Checkout(PaymentProcessor paymentProcessor) {
            this.paymentProcessor = paymentProcessor;
        }

        void payOrder(double amount) {
            paymentProcessor.pay(amount);
        }
    }
}
```

```
public static void main(String[] args) {  
    LegacyBankApi oldApi = new LegacyBankApi();  
    PaymentProcessor adapter = new BankAdapter(oldApi);  
    Checkout checkout = new Checkout(adapter);  
  
    checkout.payOrder(49.90);  
}  
}
```

Résultat obtenu à l'exécution

Ancienne banque : paiement de 4990 centimes

7. Bridge - BridgeDemo.java

Idée. Séparer deux dimensions qui peuvent évoluer indépendamment afin d'éviter une explosion de sous-classes.

Le problème. Vous avez plusieurs types de télécommandes et plusieurs types d'appareils. Avec l'héritage pur, vous pourriez finir avec BasicTvRemote, BasicRadioRemote, AdvancedTvRemote, AdvancedRadioRemote, etc.

Les rôles. RemoteControl = abstraction ; AdvancedRemoteControl = abstraction raffinée ; Device = implémentation ; Television/Radio = implémentations concrètes.

Ce qui se passe. La télécommande contient un Device et lui délègue les actions. La même AdvancedRemoteControl peut donc piloter une Television ou une Radio.

Pourquoi c'est utile. Vous pouvez faire évoluer les télécommandes et les appareils séparément, puis les combiner par composition.

Attention. Bridge ressemble parfois à Adapter visuellement. Bridge est généralement pensé dès la conception pour séparer deux axes de variation ; Adapter sert souvent à raccorder une interface existante incompatible.

À retenir : Bridge = « deux dimensions varient : je les sépare et je les relie par composition ».

Code complet à copier

```
public class BridgeDemo {

    interface Device {
        void on();
        void off();
        void setVolume(int volume);
    }

    static class Television implements Device {
        public void on() {
            System.out.println("TV allumee");
        }

        public void off() {
            System.out.println("TV eteinte");
        }

        public void setVolume(int volume) {
            System.out.println("Volume TV = " + volume);
        }
    }

    static class Radio implements Device {
        public void on() {
            System.out.println("Radio allumee");
        }

        public void off() {
            System.out.println("Radio eteinte");
        }

        public void setVolume(int volume) {
            System.out.println("Volume radio = " + volume);
        }
    }
}
```

```

static class RemoteControl {
    protected final Device device;

    RemoteControl(Device device) {
        this.device = device;
    }

    void turnOn() {
        device.on();
    }

    void turnOff() {
        device.off();
    }
}

static class AdvancedRemoteControl extends RemoteControl {
    AdvancedRemoteControl(Device device) {
        super(device);
    }

    void setVolume(int volume) {
        device.setVolume(volume);
    }
}

public static void main(String[] args) {
    AdvancedRemoteControl tvRemote = new AdvancedRemoteControl(new Television());
    tvRemote.turnOn();
    tvRemote.setVolume(25);
    tvRemote.turnOff();

    System.out.println("--- Meme telecommande, autre appareil ---");

    AdvancedRemoteControl radioRemote = new AdvancedRemoteControl(new Radio());
    radioRemote.turnOn();
    radioRemote.setVolume(8);
    radioRemote.turnOff();
}
}

```

Résultat obtenu à l'exécution

```

TV allumee
Volume TV = 25
TV eteinte
--- Meme telecommande, autre appareil ---
Radio allumee
Volume radio = 8
Radio eteinte

```

8. Composite - CompositeDemo.java

Idée. Traiter uniformément un objet individuel et un groupe d'objets organisés en arbre.

Le problème. Un système de fichiers contient des fichiers simples et des dossiers qui peuvent eux-mêmes contenir fichiers et sous-dossiers. Le client ne devrait pas avoir à écrire des instances partout.

Les rôles. FileSystemElement = composant ; File = feuille ; Folder = composite ; children = collection d'autres FileSystemElement.

Ce qui se passe. getSize() sur un File retourne sa taille. getSize() sur un Folder additionne récursivement la taille de tous ses enfants. display() suit le même principe.

Pourquoi c'est utile. Le client appelle les mêmes méthodes sur un fichier ou sur tout un dossier. La récursivité devient naturelle.

Attention. Le signe révélateur est une structure en arbre où un conteneur contient des éléments du même type abstrait que lui.

À retenir : Composite = « un objet seul et un groupe se manipulent de la même façon ».

Code complet à copier

```
import java.util.ArrayList;
import java.util.List;

public class CompositeDemo {

    interface FileSystemElement {
        int getSize();
        void display(String indent);
    }

    static class File implements FileSystemElement {
        private final String name;
        private final int size;

        File(String name, int size) {
            this.name = name;
            this.size = size;
        }

        public int getSize() {
            return size;
        }

        public void display(String indent) {
            System.out.println(indent + "- " + name + " (" + size + " Ko)");
        }
    }

    static class Folder implements FileSystemElement {
        private final String name;
        private final List<FileSystemElement> children = new ArrayList<FileSystemElement>();

        Folder(String name) {
            this.name = name;
        }

        void add(FileSystemElement element) {
            children.add(element);
        }
    }
}
```

```

    public int getSize() {
        int total = 0;
        for (FileSystemElement child : children) {
            total += child.getSize();
        }
        return total;
    }

    public void display(String indent) {
        System.out.println(indent + "+ " + name + " (" + getSize() + " Ko)");
        for (FileSystemElement child : children) {
            child.display(indent + " ");
        }
    }
}

public static void main(String[] args) {
    Folder cours = new Folder("Cours Java");
    cours.add(new File("intro.pdf", 120));
    cours.add(new File("exercice.java", 30));

    Folder corrections = new Folder("Corrections");
    corrections.add(new File("correction1.java", 40));
    corrections.add(new File("correction2.java", 50));

    cours.add(corrections);
    cours.display("");

    System.out.println("Taille totale : " + cours.getSize() + " Ko");
}

```

Résultat obtenu à l'exécution

```

+ Cours Java (240 Ko)
- intro.pdf (120 Ko)
- exercice.java (30 Ko)
+ Corrections (90 Ko)
- correction1.java (40 Ko)
- correction2.java (50 Ko)
Taille totale : 240 Ko

```

9. Decorator - DecoratorDemo.java

Idée. Ajouter dynamiquement des responsabilités à un objet en l'enveloppant, sans multiplier les sous-classes.

Le problème. Pour un café, vous pouvez ajouter lait, caramel, chantilly, etc. Une sous-classe pour chaque combinaison conduirait vite à CafeLait, CafeCaramel, CafeLaitCaramel, etc.

Les rôles. Coffee = composant ; SimpleCoffee = composant concret ; CoffeeDecorator = décorateur abstrait ; MilkDecorator/CaramelDecorator = décorateurs concrets.

Ce qui se passe. Chaque décorateur contient un Coffee. Il appelle d'abord le comportement de l'objet enveloppé puis ajoute son propre prix et sa propre description.

Pourquoi c'est utile. Les options deviennent combinables à l'exécution : vous pouvez empiler les décorateurs dans l'ordre souhaité.

Attention. Decorator conserve la même interface que l'objet décoré. Il enrichit le comportement ; il ne remplace pas simplement l'interface comme Adapter.

À retenir : Decorator = « j'enveloppe l'objet pour lui ajouter des capacités ».

Code complet à copier

```
public class DecoratorDemo {

    interface Coffee {
        String description();
        double price();
    }

    static class SimpleCoffee implements Coffee {
        public String description() {
            return "Cafe";
        }

        public double price() {
            return 2.00;
        }
    }

    static abstract class CoffeeDecorator implements Coffee {
        protected final Coffee coffee;

        CoffeeDecorator(Coffee coffee) {
            this.coffee = coffee;
        }
    }

    static class MilkDecorator extends CoffeeDecorator {
        MilkDecorator(Coffee coffee) {
            super(coffee);
        }

        public String description() {
            return coffee.description() + " + lait";
        }

        public double price() {
            return coffee.price() + 0.50;
        }
    }
}
```

```

static class CaramelDecorator extends CoffeeDecorator {
    CaramelDecorator(Coffee coffee) {
        super(coffee);
    }

    public String description() {
        return coffee.description() + " + caramel";
    }

    public double price() {
        return coffee.price() + 0.70;
    }
}

public static void main(String[] args) {
    Coffee coffee = new SimpleCoffee();
    coffee = new MilkDecorator(coffee);
    coffee = new CaramelDecorator(coffee);

    System.out.println(coffee.description());
    System.out.printf("Prix : %.2f euros\n", coffee.price());
}
}

```

Résultat obtenu à l'exécution

```

Cafe + lait + caramel
Prix : 3.20 euros

```

10. Facade - FacadeDemo.java

Idée. Fournir une entrée simple vers un sous-système composé de plusieurs services.

Le problème. Passer une commande demande de vérifier le stock, encaisser, créer une livraison et envoyer un e-mail. Le client ne devrait pas orchestrer lui-même tout ce protocole.

Les rôles. OrderFacade = façade ; StockService, PaymentService, DeliveryService, EmailService = sous-système ; main = client.

Ce qui se passe. Le client appelle une seule méthode placeOrder(). La façade coordonne les services dans le bon ordre.

Pourquoi c'est utile. Le client connaît moins de classes et moins de détails de protocole. L'usage courant devient plus simple.

Attention. Facade ne rend pas forcément les classes internes inaccessibles. Elle fournit surtout une voie simple et stable pour les scénarios courants.

À retenir : Facade = « un point d'entrée simple devant un système complexe ».

Code complet à copier

```
public class FacadeDemo {

    static class StockService {
        boolean isAvailable(String product) {
            System.out.println("Stock verifie pour : " + product);
            return true;
        }
    }

    static class PaymentService {
        void pay(double amount) {
            System.out.println("Paieement accepte : " + amount + " euros");
        }
    }

    static class DeliveryService {
        void createDelivery(String product) {
            System.out.println("Livraison creee pour : " + product);
        }
    }

    static class EmailService {
        void sendConfirmation() {
            System.out.println("Email de confirmation envoye");
        }
    }

    static class OrderFacade {
        private final StockService stock = new StockService();
        private final PaymentService payment = new PaymentService();
        private final DeliveryService delivery = new DeliveryService();
        private final EmailService email = new EmailService();

        void placeOrder(String product, double amount) {
            if (!stock.isAvailable(product)) {
                throw new IllegalStateException("Produit indisponible");
            }
            payment.pay(amount);
            delivery.createDelivery(product);
            email.sendConfirmation();
        }
    }
}
```

```
public static void main(String[] args) {  
    OrderFacade facade = new OrderFacade();  
    facade.placeOrder("PC portable", 1299.00);  
}
```

Résultat obtenu à l'exécution

Stock verifie pour : PC portable
Palement accepte : 1299.0 euros
Livraison creee pour : PC portable
Email de confirmation envoye

11. Flyweight - FlyweightDemo.java

Idée. Partager les données lourdes identiques entre un grand nombre d'objets afin d'économiser de la mémoire.

Le problème. Une forêt peut contenir des milliers d'arbres. Le nom, la couleur et la texture d'un chêne sont identiques pour beaucoup d'arbres, alors que leurs coordonnées x/y changent.

Les rôles. TreeType = état intrinsèque partagé ; Tree = état extrinsèque x/y ; TreeTypeFactory = cache des flyweights.

Ce qui se passe. Deux Tree demandent le même TreeType au cache. La fabrique ne crée qu'un seul objet TreeType et le partage entre les deux arbres.

Pourquoi c'est utile. Les données répétées ne sont stockées qu'une fois. Les objets individuels gardent seulement leurs données propres.

Attention. Le pattern n'est utile que si le nombre d'objets est élevé et qu'une part significative de leur état peut réellement être partagée.

À retenir : Flyweight = « je mutualise l'état identique entre beaucoup de petits objets ».

Code complet à copier

```
import java.util.HashMap;
import java.util.Map;

public class FlyweightDemo {

    static class TreeType {
        private final String name;
        private final String color;
        private final String texture;

        TreeType(String name, String color, String texture) {
            this.name = name;
            this.color = color;
            this.texture = texture;
        }

        void draw(int x, int y) {
            System.out.println(name + " " + color + " " + texture
                + " dessine en (" + x + ", " + y + ")");
        }
    }

    static class TreeTypeFactory {
        private static final Map<String, TreeType> CACHE = new HashMap<String, TreeType>();

        static TreeType getTreeType(String name, String color, String texture) {
            String key = name + "|" + color + "|" + texture;
            TreeType type = CACHE.get(key);
            if (type == null) {
                type = new TreeType(name, color, texture);
                CACHE.put(key, type);
                System.out.println("Creation d'un TreeType partage : " + key);
            }
            return type;
        }

        static int cacheSize() {
            return CACHE.size();
        }
    }
}
```

```

}

static class Tree {
    private final int x;
    private final int y;
    private final TreeType type;

    Tree(int x, int y, TreeType type) {
        this.x = x;
        this.y = y;
        this.type = type;
    }

    void draw() {
        type.draw(x, y);
    }
}

public static void main(String[] args) {
    TreeType oak = TreeTypeFactory.getTreeType("Chene", "vert", "rugueuse");
    Tree tree1 = new Tree(10, 20, oak);
    Tree tree2 = new Tree(30, 40,
        TreeTypeFactory.getTreeType("Chene", "vert", "rugueuse"));

    tree1.draw();
    tree2.draw();

    System.out.println("Nombre d'objets TreeType crees : " + TreeTypeFactory.cacheSize());
    System.out.println("Type partage ? " + (tree1.type == tree2.type));
}
}

```

Résultat obtenu à l'exécution

```

Creation d'un TreeType partage : Chene|vert|rugueuse
Chene vert rugueuse dessine en (10, 20)
Chene vert rugueuse dessine en (30, 40)
Nombre d'objets TreeType crees : 1
Type partage ? true

```

12. Proxy - ProxyDemo.java

Idée. Placer un objet intermédiaire qui contrôle l'accès à l'objet réel tout en exposant la même interface.

Le problème. Charger une image 4K depuis le disque coûte cher. Vous voulez créer un objet immédiatement mais ne charger réellement l'image que lorsqu'elle doit être affichée.

Les rôles. Image = interface commune ; RealImage = objet réel coûteux ; ImageProxy = proxy paresseux.

Ce qui se passe. La création de ImageProxy ne charge rien. Au premier display(), le proxy crée RealImage. Au second display(), il réutilise l'objet déjà chargé.

Pourquoi c'est utile. Un proxy peut gérer lazy loading, contrôle d'accès, cache, journalisation ou accès distant sans changer le code client.

Attention. Proxy et Decorator ont souvent une structure similaire. Le Proxy contrôle l'accès ; le Decorator ajoute des responsabilités fonctionnelles.

À retenir : Proxy = « je mets un représentant devant l'objet réel pour contrôler son accès ».

Code complet à copier

```
public class ProxyDemo {

    interface Image {
        void display();
    }

    static class RealImage implements Image {
        private final String fileName;

        RealImage(String fileName) {
            this.fileName = fileName;
            loadFromDisk();
        }

        private void loadFromDisk() {
            System.out.println("Chargement couteux de " + fileName);
        }

        public void display() {
            System.out.println("Affichage de " + fileName);
        }
    }

    static class ImageProxy implements Image {
        private final String fileName;
        private RealImage realImage;

        ImageProxy(String fileName) {
            this.fileName = fileName;
        }

        public void display() {
            if (realImage == null) {
                realImage = new RealImage(fileName);
            }
            realImage.display();
        }
    }

    public static void main(String[] args) {
        Image image = new ImageProxy("photo-4k.jpg");
    }
}
```

```
        System.out.println("Proxy cree : l'image n'est pas encore chargee.");  
        image.display();  
        System.out.println("Deuxieme affichage : pas de nouveau chargement.");  
        image.display();  
    }  
}
```

Résultat obtenu à l'exécution

Proxy cree : l'image n'est pas encore chargee.
Chargement couteux de photo-4k.jpg
Affichage de photo-4k.jpg
Deuxieme affichage : pas de nouveau chargement.
Affichage de photo-4k.jpg

Patterns de comportement

13. Chain of Responsibility - ChainOfResponsibilityDemo.java

Idée. Faire circuler une requête le long d'une chaîne jusqu'à ce qu'un objet décide de la traiter.

Le problème. Un support technique possède plusieurs niveaux. Le premier niveau traite les problèmes simples, le deuxième les problèmes intermédiaires et l'expert les problèmes complexes.

Les rôles. SupportHandler = handler abstrait ; Level1Support, Level2Support, ExpertSupport = handlers concrets ; next = maillon suivant.

Ce qui se passe. handle() demande d'abord canHandle(). Si le handler ne peut pas traiter, il délègue à next. Le client envoie toujours la requête au premier maillon.

Pourquoi c'est utile. L'émetteur ne connaît pas le destinataire final. Vous pouvez réordonner ou compléter la chaîne sans réécrire le client.

Attention. Il faut définir clairement ce qui se passe si aucun maillon ne traite la requête et éviter des chaînes opaques trop longues.

À retenir : Chain of Responsibility = « si je ne peux pas traiter, je passe au suivant ».

Code complet à copier

```
public class ChainOfResponsibilityDemo {

    static abstract class SupportHandler {
        private SupportHandler next;

        SupportHandler setNext(SupportHandler next) {
            this.next = next;
            return next;
        }

        void handle(int level, String problem) {
            if (canHandle(level)) {
                process(problem);
            } else if (next != null) {
                next.handle(level, problem);
            } else {
                System.out.println("Aucun niveau ne peut traiter : " + problem);
            }
        }

        protected abstract boolean canHandle(int level);
        protected abstract void process(String problem);
    }

    static class Level1Support extends SupportHandler {
        protected boolean canHandle(int level) {
            return level == 1;
        }

        protected void process(String problem) {
            System.out.println("Support niveau 1 traite : " + problem);
        }
    }

    static class Level2Support extends SupportHandler {
        protected boolean canHandle(int level) {
```

```

        return level == 2;
    }

    protected void process(String problem) {
        System.out.println("Support niveau 2 traite : " + problem);
    }
}

static class ExpertSupport extends SupportHandler {
    protected boolean canHandle(int level) {
        return level >= 3;
    }

    protected void process(String problem) {
        System.out.println("Expert traite : " + problem);
    }
}

public static void main(String[] args) {
    SupportHandler level1 = new Level1Support();
    level1.setNext(new Level2Support())
        .setNext(new ExpertSupport());

    level1.handle(1, "Mot de passe oublie");
    level1.handle(2, "Application tres lente");
    level1.handle(3, "Base de donnees corrompue");
}
}

```

Résultat obtenu à l'exécution

```

Support niveau 1 traite : Mot de passe oublie
Support niveau 2 traite : Application tres lente
Expert traite : Base de donnees corrompue

```

14. Command - CommandDemo.java

Idée. Transformer une action en objet afin de pouvoir la stocker, la transmettre, l'historiser ou l'annuler.

Le problème. Une télécommande doit déclencher des actions sans connaître leurs détails, et vous voulez pouvoir annuler la dernière action.

Les rôles. Command = interface de commande ; LightOnCommand = commande concrète ; Light = receiver ; RemoteControl = invoker.

Ce qui se passe. RemoteControl.press() exécute la commande et l'empile dans l'historique. undoLast() récupère la dernière commande et appelle undo().

Pourquoi c'est utile. Une opération devient une donnée manipulable. Cela facilite historique, undo/redo, files de tâches et macros.

Attention. La commande encapsule l'intention d'action. Elle ne doit pas forcément contenir toute la logique métier ; elle peut déléguer au receiver.

À retenir : Command = « une action devient un objet ».

Code complet à copier

```
import java.util.ArrayDeque;
import java.util.Deque;

public class CommandDemo {

    interface Command {
        void execute();
        void undo();
    }

    static class Light {
        void on() {
            System.out.println("Lampe allumee");
        }

        void off() {
            System.out.println("Lampe eteinte");
        }
    }

    static class LightOnCommand implements Command {
        private final Light light;

        LightOnCommand(Light light) {
            this.light = light;
        }

        public void execute() {
            light.on();
        }

        public void undo() {
            light.off();
        }
    }

    static class RemoteControl {
        private final Deque<Command> history = new ArrayDeque<Command>();
    }
}
```

```

    void press(Command command) {
        command.execute();
        history.push(command);
    }

    void undoLast() {
        if (!history.isEmpty()) {
            history.pop().undo();
        }
    }
}

public static void main(String[] args) {
    Light livingRoomLight = new Light();
    Command turnOn = new LightOnCommand(livingRoomLight);

    RemoteControl remote = new RemoteControl();
    remote.press(turnOn);
    remote.undoLast();
}
}

```

Résultat obtenu à l'exécution

```

Lampe allumee
Lampe eteinte

```

15. Interpreter - InterpreterDemo.java

Idée. Représenter les règles d'un petit langage par des objets et interpréter une expression construite avec ces règles.

Le problème. Vous voulez évaluer une mini-expression arithmétique composée de nombres, variables, additions et multiplications.

Les rôles. Expression = expression abstraite ; NumberExpression/VariableExpression = terminaux ; AddExpression/MultiplyExpression = non-terminaux ; context = valeurs des variables.

Ce qui se passe. L'objet représentant $(x + 3) * 2$ forme un arbre. interpret() descend récursivement dans cet arbre et calcule le résultat.

Pourquoi c'est utile. Chaque règle grammaticale est isolée dans une classe. Pour de petits DSL, le modèle est très pédagogique et extensible.

Attention. Pour un vrai langage complexe, ce pattern devient vite lourd ; un parser ou un générateur de grammaire est souvent plus adapté.

À retenir : Interpreter = « ma grammaire devient un arbre d'objets que j'évalue ».

Code complet à copier

```
import java.util.HashMap;
import java.util.Map;

public class InterpreterDemo {

    interface Expression {
        int interpret(Map<String, Integer> context);
    }

    static class NumberExpression implements Expression {
        private final int value;

        NumberExpression(int value) {
            this.value = value;
        }

        public int interpret(Map<String, Integer> context) {
            return value;
        }
    }

    static class VariableExpression implements Expression {
        private final String name;

        VariableExpression(String name) {
            this.name = name;
        }

        public int interpret(Map<String, Integer> context) {
            Integer value = context.get(name);
            if (value == null) {
                throw new IllegalArgumentException("Variable inconnue : " + name);
            }
            return value;
        }
    }

    static class AddExpression implements Expression {
```

```

    private final Expression left;
    private final Expression right;

    AddExpression(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret(Map<String, Integer> context) {
        return left.interpret(context) + right.interpret(context);
    }
}

static class MultiplyExpression implements Expression {
    private final Expression left;
    private final Expression right;

    MultiplyExpression(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    public int interpret(Map<String, Integer> context) {
        return left.interpret(context) * right.interpret(context);
    }
}

public static void main(String[] args) {
    Map<String, Integer> context = new HashMap<String, Integer>();
    context.put("x", 5);

    // Expression representee : (x + 3) * 2
    Expression expression = new MultiplyExpression(
        new AddExpression(
            new VariableExpression("x"),
            new NumberExpression(3)
        ),
        new NumberExpression(2)
    );

    System.out.println("(x + 3) * 2 = " + expression.interpret(context));
}
}

```

Résultat obtenu à l'exécution

```
(x + 3) * 2 = 16
```

16. Iterator - IteratorDemo.java

Idée. Parcourir une collection sans exposer sa représentation interne.

Le problème. Vous voulez permettre un for-each sur une collection personnalisée de noms, sans donner au client l'accès direct à l'index interne.

Les rôles. NameCollection = agrégat ; NameIterator = itérateur ; Iterator<String> = contrat standard Java.

Ce qui se passe. NameCollection implémente Iterable<String>. Le for-each appelle automatiquement iterator(), puis hasNext() et next().

Pourquoi c'est utile. Le client connaît seulement le protocole de parcours, pas la structure de stockage.

Attention. Java fournit déjà Iterator et Iterable. Le pattern reste important à comprendre car for-each repose directement sur ce principe.

À retenir : Iterator = « parcourir sans connaître la structure interne ».

Code complet à copier

```
import java.util.Iterator;
import java.util.NoSuchElementException;

public class IteratorDemo {

    static class NameCollection implements Iterable<String> {
        private final String[] names;

        NameCollection(String... names) {
            this.names = names;
        }

        public Iterator<String> iterator() {
            return new NameIterator();
        }

        private class NameIterator implements Iterator<String> {
            private int index = 0;

            public boolean hasNext() {
                return index < names.length;
            }

            public String next() {
                if (!hasNext()) {
                    throw new NoSuchElementException();
                }
                return names[index++];
            }
        }
    }

    public static void main(String[] args) {
        NameCollection students = new NameCollection("Alice", "Bob", "Chloe");

        for (String student : students) {
            System.out.println(student);
        }
    }
}
```

Résultat obtenu à l'exécution

Alice
Bob
Chloe

17. Mediator - MediatorDemo.java

Idée. Centraliser les échanges entre plusieurs objets afin qu'ils ne se référencent pas tous directement entre eux.

Le problème. Dans un salon de discussion, si chaque utilisateur devait connaître tous les autres utilisateurs, le couplage augmenterait très vite.

Les rôles. ChatMediator = contrat du médiateur ; ChatRoom = médiateur concret ; User = collègues.

Ce qui se passe. Alice envoie un message au ChatRoom. Le médiateur décide à quels utilisateurs le transmettre. Alice ne connaît ni Bob ni Chloé.

Pourquoi c'est utile. Les relations N-vers-N sont remplacées par des relations de chaque collègue vers un médiateur central.

Attention. Le médiateur peut devenir une « grosse classe centrale » s'il absorbe trop de logique. Il doit surtout orchestrer les interactions.

À retenir : Mediator = « les objets ne se parlent plus directement ; ils passent par un coordinateur ».

Code complet à copier

```
import java.util.ArrayList;
import java.util.List;

public class MediatorDemo {

    interface ChatMediator {
        void register(User user);
        void send(String message, User sender);
    }

    static class ChatRoom implements ChatMediator {
        private final List<User> users = new ArrayList<User>();

        public void register(User user) {
            users.add(user);
        }

        public void send(String message, User sender) {
            for (User user : users) {
                if (user != sender) {
                    user.receive(sender.name + " : " + message);
                }
            }
        }
    }

    static class User {
        private final String name;
        private final ChatMediator mediator;

        User(String name, ChatMediator mediator) {
            this.name = name;
            this.mediator = mediator;
            mediator.register(this);
        }

        void send(String message) {
            System.out.println(name + " envoie : " + message);
            mediator.send(message, this);
        }
    }
}
```

```

    void receive(String message) {
        System.out.println(name + " recoit -> " + message);
    }
}

public static void main(String[] args) {
    ChatMediator room = new ChatRoom();

    User alice = new User("Alice", room);
    User bob = new User("Bob", room);
    User chloe = new User("Chloe", room);

    alice.send("Bonjour tout le monde !");
}
}

```

Résultat obtenu à l'exécution

```

Alice envoie : Bonjour tout le monde !
Bob recoit -> Alice : Bonjour tout le monde !
Chloe recoit -> Alice : Bonjour tout le monde !

```

18. Memento - MementoDemo.java

Idée. Capturer l'état d'un objet afin de pouvoir le restaurer plus tard sans exposer ses détails internes.

Le problème. Un éditeur de texte doit permettre un undo. Vous devez mémoriser l'ancien contenu avant une modification.

Les rôles. `TextEditor` = originator ; `TextEditor.Memento` = instantané ; `History` = caretaker.

Ce qui se passe. `editor.save()` crée un `Memento` avec l'état courant. `History` le stocke. Plus tard, `editor.restore()` réinjecte cet état.

Pourquoi c'est utile. La logique de sauvegarde/restauration appartient à l'objet qui connaît son propre état, tandis que l'historique manipule des snapshots opaques.

Attention. Stocker beaucoup de gros snapshots peut coûter cher en mémoire. Il faut parfois utiliser des deltas ou limiter l'historique.

À retenir : Memento = « je prends une photo de l'état pour pouvoir revenir en arrière ».

Code complet à copier

```
import java.util.ArrayDeque;
import java.util.Deque;

public class MementoDemo {

    static class TextEditor {
        private String text = "";

        void setText(String text) {
            this.text = text;
        }

        String getText() {
            return text;
        }

        Memento save() {
            return new Memento(text);
        }

        void restore(Memento memento) {
            this.text = memento.text;
        }

        static class Memento {
            private final String text;

            private Memento(String text) {
                this.text = text;
            }
        }
    }

    static class History {
        private final Deque<TextEditor.Memento> states = new ArrayDeque<TextEditor.Memento>();

        void push(TextEditor.Memento memento) {
            states.push(memento);
        }
    }
}
```

```

        TextEditor.Memento pop() {
            return states.pop();
        }
    }

    public static void main(String[] args) {
        TextEditor editor = new TextEditor();
        History history = new History();

        editor.setText("Version 1");
        history.push(editor.save());

        editor.setText("Version 2");
        System.out.println("Avant undo : " + editor.getText());

        editor.restore(history.pop());
        System.out.println("Après undo : " + editor.getText());
    }
}

```

Résultat obtenu à l'exécution

```

Avant undo : Version 2
Après undo : Version 1

```

19. Observer - ObserverDemo.java

Idée. Notifier automatiquement plusieurs objets lorsqu'un sujet change d'état.

Le problème. Une station météo doit prévenir plusieurs écrans dès qu'une nouvelle température est disponible, sans connaître leur logique interne.

Les rôles. Observer = contrat des abonnés ; WeatherStation = sujet ; PhoneDisplay = observateur concret.

Ce qui se passe. Les écrans s'abonnent avec subscribe(). Lorsque setTemperature() change l'état, notifyObservers() appelle update() sur chaque observateur.

Pourquoi c'est utile. Le sujet ne dépend que de l'interface Observer. Vous pouvez ajouter un écran web ou une alerte sans modifier WeatherStation.

Attention. Pensez au désabonnement et aux références persistantes. Dans de gros systèmes, les chaînes de notifications peuvent être difficiles à suivre.

À retenir : Observer = « quand je change, je préviens mes abonnés ».

Code complet à copier

```
import java.util.ArrayList;
import java.util.List;

public class ObserverDemo {

    interface Observer {
        void update(int temperature);
    }

    static class WeatherStation {
        private final List<Observer> observers = new ArrayList<Observer>();
        private int temperature;

        void subscribe(Observer observer) {
            observers.add(observer);
        }

        void unsubscribe(Observer observer) {
            observers.remove(observer);
        }

        void setTemperature(int temperature) {
            this.temperature = temperature;
            notifyObservers();
        }

        private void notifyObservers() {
            for (Observer observer : observers) {
                observer.update(temperature);
            }
        }
    }

    static class PhoneDisplay implements Observer {
        private final String owner;

        PhoneDisplay(String owner) {
            this.owner = owner;
        }
    }
}
```

```

        public void update(int temperature) {
            System.out.println(owner + " recoit : temperature = " + temperature + " C");
        }

    public static void main(String[] args) {
        WeatherStation station = new WeatherStation();

        Observer phoneAlice = new PhoneDisplay("Alice");
        Observer phoneBob = new PhoneDisplay("Bob");

        station.subscribe(phoneAlice);
        station.subscribe(phoneBob);

        station.setTemperature(24);
        station.setTemperature(27);
    }
}

```

Résultat obtenu à l'exécution

```

Alice recoit : temperature = 24 C
Bob recoit : temperature = 24 C
Alice recoit : temperature = 27 C
Bob recoit : temperature = 27 C

```

20. State - StateDemo.java

Idée. Faire varier le comportement d'un objet en fonction de son état interne en représentant chaque état par un objet.

Le problème. Un distributeur automatique ne réagit pas de la même façon à selectProduct() selon qu'une pièce est présente ou non. De nombreux if(state == ...) finirait par envahir la classe.

Les rôles. State = contrat ; WaitingForCoinState/HasCoinState = états concrets ; VendingMachine = contexte.

Ce qui se passe. VendingMachine délègue l'action à son état courant. L'état peut ensuite appeler machine.setState(...) pour provoquer une transition.

Pourquoi c'est utile. Chaque comportement lié à un état est regroupé dans sa classe. Les transitions deviennent explicites.

Attention. State ressemble structurellement à Strategy, mais l'intention diffère : Strategy représente un algorithme choisi ; State représente le mode courant d'un objet et les transitions de son cycle de vie.

À retenir : State = « mon comportement change parce que mon état a changé ».

Code complet à copier

```
public class StateDemo {

    interface State {
        void insertCoin(VendingMachine machine);
        void selectProduct(VendingMachine machine);
    }

    static class WaitingForCoinState implements State {
        public void insertCoin(VendingMachine machine) {
            System.out.println("Piece acceptee.");
            machine.setState(new HasCoinState());
        }

        public void selectProduct(VendingMachine machine) {
            System.out.println("Inserez d'abord une piece.");
        }
    }

    static class HasCoinState implements State {
        public void insertCoin(VendingMachine machine) {
            System.out.println("Une piece est deja inseree.");
        }

        public void selectProduct(VendingMachine machine) {
            System.out.println("Produit distribue.");
            machine.setState(new WaitingForCoinState());
        }
    }

    static class VendingMachine {
        private State state = new WaitingForCoinState();

        void setState(State state) {
            this.state = state;
        }

        void insertCoin() {
            state.insertCoin(this);
        }
    }
}
```

```

    }

    void selectProduct() {
        state.selectProduct(this);
    }
}

public static void main(String[] args) {
    VendingMachine machine = new VendingMachine();

    machine.selectProduct();
    machine.insertCoin();
    machine.insertCoin();
    machine.selectProduct();
    machine.selectProduct();
}
}

```

Résultat obtenu à l'exécution

```

Inserez d'abord une piece.
Piece acceptee.
Une piece est deja inseree.
Produit distribue.
Inserez d'abord une piece.

```

21. Strategy - StrategyDemo.java

Idée. Encapsuler plusieurs algorithmes interchangeables derrière une même interface.

Le problème. Un GPS peut calculer un trajet voiture, vélo ou piéton. Vous ne voulez pas mettre chaque algorithme complet dans un immense if/else du GPS.

Les rôles. RouteStrategy = stratégie ; CarStrategy/BikeStrategy/WalkingStrategy = stratégies concrètes ; GPS = contexte.

Ce qui se passe. Le choix du mode est réalisé à l'extérieur du GPS, ici via une Map. GPS reçoit une RouteStrategy et délègue calculateRoute() à cette stratégie. Il peut même changer de stratégie à l'exécution.

Pourquoi c'est utile. Vous séparez « quelle stratégie choisir ? » de « comment l'algorithme fonctionne ? ». Le GPS reste stable lorsque vous ajoutez un nouveau mode.

Attention. Strategy ne supprime pas magiquement toute décision. Il faut toujours choisir une stratégie quelque part. Le gain est que le code métier ne contient plus les différents algorithmes dans ses branches conditionnelles.

À retenir : Strategy = « même tâche, plusieurs façons de la réaliser ; je choisis l'algorithme ».

Code complet à copier

```
import java.util.HashMap;
import java.util.Map;

public class StrategyDemo {

    interface RouteStrategy {
        void calculate(String from, String to);
    }

    static class CarStrategy implements RouteStrategy {
        public void calculate(String from, String to) {
            System.out.println("Voiture : routes et autoroutes de " + from + " a " + to);
        }
    }

    static class BikeStrategy implements RouteStrategy {
        public void calculate(String from, String to) {
            System.out.println("Velo : priorite aux pistes cyclables de " + from + " a " + to);
        }
    }

    static class WalkingStrategy implements RouteStrategy {
        public void calculate(String from, String to) {
            System.out.println("Pieton : chemins pietons de " + from + " a " + to);
        }
    }

    static class GPS {
        private RouteStrategy strategy;

        GPS(RouteStrategy strategy) {
            this.strategy = strategy;
        }

        void setStrategy(RouteStrategy strategy) {
            this.strategy = strategy;
        }
    }
}
```

```

        void calculateRoute(String from, String to) {
            strategy.calculate(from, to);
        }
    }

    public static void main(String[] args) {
        Map<String, RouteStrategy> strategies = new HashMap<String, RouteStrategy>();
        strategies.put("voiture", new CarStrategy());
        strategies.put("velo", new BikeStrategy());
        strategies.put("pieton", new WalkingStrategy());

        String requestedMode = "velo";
        RouteStrategy selected = strategies.get(requestedMode);
        if (selected == null) {
            throw new IllegalArgumentException("Mode inconnu : " + requestedMode);
        }

        GPS gps = new GPS(selected);
        gps.calculateRoute("Paris", "Versailles");

        gps.setStrategy(strategies.get("pieton"));
        gps.calculateRoute("Gare", "Hotel");
    }
}

```

Résultat obtenu à l'exécution

Velo : priorite aux pistes cyclables de Paris a Versailles
 Pieton : chemins pietons de Gare a Hotel

22. Template Method - TemplateMethodDemo.java

Idée. Définir le squelette fixe d'un algorithme dans une classe mère et laisser certaines étapes variables aux sous-classes.

Le problème. Importer du CSV ou du JSON suit toujours la même séquence : ouvrir, lire, valider, sauvegarder, fermer. Seules lecture et validation diffèrent.

Les rôles. DataImporter = classe abstraite contenant le template ; importData() = template method ; CsvImporter/JsonImporter = sous-classes qui redéfinissent les étapes variables.

Ce qui se passe. importData() impose l'ordre. Les appels readData() et validateData() sont résolus polymorphiquement selon la sous-classe concrète.

Pourquoi c'est utile. Vous réutilisez la structure commune de l'algorithme tout en laissant quelques points d'extension contrôlés.

Attention. Template Method repose sur l'héritage, alors que Strategy repose plutôt sur la composition. Strategy permet généralement de changer d'algorithme à l'exécution plus facilement.

À retenir : Template Method = « le scénario est fixe, certaines étapes sont personnalisables ».

Code complet à copier

```
public class TemplateMethodDemo {

    static abstract class DataImporter {
        public final void importData() {
            openFile();
            readData();
            validateData();
            saveData();
            closeFile();
        }

        private void openFile() {
            System.out.println("1. Ouverture du fichier");
        }

        protected abstract void readData();
        protected abstract void validateData();

        private void saveData() {
            System.out.println("4. Sauvegarde en base");
        }

        private void closeFile() {
            System.out.println("5. Fermeture du fichier");
        }
    }

    static class CsvImporter extends DataImporter {
        protected void readData() {
            System.out.println("2. Lecture des lignes CSV");
        }

        protected void validateData() {
            System.out.println("3. Validation des colonnes CSV");
        }
    }

    static class JsonImporter extends DataImporter {
```

```

        protected void readData() {
            System.out.println("2. Lecture des objets JSON");
        }

        protected void validateData() {
            System.out.println("3. Validation du schema JSON");
        }
    }

    public static void main(String[] args) {
        DataImporter importer = new CsvImporter();
        importer.importData();

        System.out.println("--- JSON ---");

        importer = new JsonImporter();
        importer.importData();
    }
}

```

Résultat obtenu à l'exécution

```

1. Ouverture du fichier
2. Lecture des lignes CSV
3. Validation des colonnes CSV
4. Sauvegarde en base
5. Fermeture du fichier
--- JSON ---
1. Ouverture du fichier
2. Lecture des objets JSON
3. Validation du schema JSON
4. Sauvegarde en base
5. Fermeture du fichier

```

23. Visitor - VisitorDemo.java

Idée. Ajouter de nouvelles opérations à une hiérarchie d'objets sans placer toutes ces opérations dans les classes métier.

Le problème. Vous avez Circle et Rectangle. Vous voulez calculer les aires, exporter, générer des statistiques, etc. Ajouter toutes ces méthodes dans chaque forme peut alourdir la hiérarchie.

Les rôles. Shape = élément ; Circle/Rectangle = éléments concrets ; ShapeVisitor = visiteur ; AreaVisitor/ExportVisitor = visiteurs concrets.

Ce qui se passe. shape.accept(visitor) appelle visitor.visit(this). Grâce à cette double dispatch, la méthode visit spécifique au type concret Circle ou Rectangle est sélectionnée.

Pourquoi c'est utile. Ajouter une nouvelle opération revient à ajouter un nouveau Visitor, sans modifier chaque algorithme existant dans les formes.

Attention. Visitor est excellent lorsque la hiérarchie d'éléments est stable et que les opérations changent souvent. Si vous ajoutez souvent de nouveaux types d'éléments, vous devrez modifier tous les visiteurs.

À retenir : Visitor = « mes types sont stables, mais j'ajoute souvent de nouvelles opérations dessus ».

Code complet à copier

```
public class VisitorDemo {

    interface Shape {
        void accept(ShapeVisitor visitor);
    }

    static class Circle implements Shape {
        final double radius;

        Circle(double radius) {
            this.radius = radius;
        }

        public void accept(ShapeVisitor visitor) {
            visitor.visit(this);
        }
    }

    static class Rectangle implements Shape {
        final double width;
        final double height;

        Rectangle(double width, double height) {
            this.width = width;
            this.height = height;
        }

        public void accept(ShapeVisitor visitor) {
            visitor.visit(this);
        }
    }

    interface ShapeVisitor {
        void visit(Circle circle);
        void visit(Rectangle rectangle);
    }
}
```

```

static class AreaVisitor implements ShapeVisitor {
    public void visit(Circle circle) {
        double area = Math.PI * circle.radius * circle.radius;
        System.out.printf("Aire du cercle : %.2f%n", area);
    }

    public void visit(Rectangle rectangle) {
        double area = rectangle.width * rectangle.height;
        System.out.printf("Aire du rectangle : %.2f%n", area);
    }
}

static class ExportVisitor implements ShapeVisitor {
    public void visit(Circle circle) {
        System.out.println("Export cercle : radius=" + circle.radius);
    }

    public void visit(Rectangle rectangle) {
        System.out.println("Export rectangle : " + rectangle.width + "x" + rectangle.height);
    }
}

public static void main(String[] args) {
    Shape[] shapes = {
        new Circle(3),
        new Rectangle(4, 5)
    };

    ShapeVisitor areaVisitor = new AreaVisitor();
    ShapeVisitor exportVisitor = new ExportVisitor();

    for (Shape shape : shapes) {
        shape.accept(areaVisitor);
        shape.accept(exportVisitor);
    }
}

```

Résultat obtenu à l'exécution

```

Aire du cercle : 28.27
Export cercle : radius=3.0
Aire du rectangle : 20.00
Export rectangle : 4.0x5.0

```

Tableau de reconnaissance rapide

Pattern	Signal typique
Factory Method	Vous voulez déléguer le choix d'un type concret à un créateur spécialisé.
Abstract Factory	Vous devez créer plusieurs objets appartenant à la même famille.
Builder	Un objet possède beaucoup d'options ou une construction en plusieurs étapes.
Prototype	Vous voulez créer une variante en copiant un modèle existant.
Singleton	Vous voulez contrôler l'existence d'une instance unique.
Adapter	Deux interfaces incompatibles doivent collaborer.
Bridge	Deux dimensions indépendantes varient et l'héritage exploserait.
Composite	Vous manipulez une arborescence de feuilles et de groupes.
Decorator	Vous voulez ajouter dynamiquement des options à un objet.
Facade	Vous voulez simplifier l'accès à plusieurs services.
Flyweight	Vous avez énormément d'objets partageant beaucoup de données identiques.
Proxy	Vous voulez contrôler ou retarder l'accès à un objet réel.
Chain of Responsibility	Plusieurs objets peuvent traiter une requête ; on essaie successivement.
Command	Vous voulez représenter une action comme un objet, notamment pour undo/redo.
Interpreter	Vous implémentez un petit langage ou une petite grammaire.
Iterator	Vous voulez parcourir une collection sans révéler son stockage.
Mediator	De nombreux objets communiquent trop directement entre eux.
Memento	Vous voulez sauvegarder puis restaurer un état.
Observer	Plusieurs abonnés doivent être prévenus d'un changement.
State	Le comportement change selon l'état courant de l'objet.
Strategy	Plusieurs algorithmes interchangeables réalisent la même tâche.
Template Method	Le squelette d'un algorithme est fixe, quelques étapes varient.
Visitor	Les types sont stables mais vous ajoutez régulièrement de nouvelles opérations.

Conseil pédagogique : ne cherchez pas d'abord le nom du pattern. Repérez le problème de conception, les responsabilités qui changent, puis demandez-vous quel découplage vous voulez obtenir.